

Princípios de Programação

Exercícios

Universidade de Lisboa
Faculdade de Ciências
Departamento de Informática
Licenciatura em Engenharia Informática

2023/2024

Módulos

1. Escreva um módulo que implemente um novo tipo `Set`, que represente um conjunto ordenado de elementos que sejam **Ord**. Como representação do conjunto utilize listas ordenadas sem elementos repetidos. Considere as seguintes operações:

- `empty`, criação de um conjunto vazio,
- `singleton`, criação de um conjunto singular,
- **`null`**, o dado conjunto é vazio?
- `size`, o número de elementos no conjunto,
- `member`, está um elemento em um conjunto?
- **`insert`**, inserção de um elemento num conjunto,
- `remove`, remoção de um elemento num conjunto,
- **`union`**, união de dois conjuntos,
- `intersection`, intersecção de dois conjuntos,
- `difference`, o conjunto diferença,
- **`filter`**, filtrar os elementos que satisfazem o predicado,
- **`partition`**, dividir o conjunto em dois, aqueles que satisfazem o predicado e aqueles que não,
- `fromList`, um conjunto com os elementos constantes numa lista.

Até que ponto as seguintes funções seriam úteis na definição de muitas destas operações?

```
liftS :: ([a] -> [b]) -> Set a -> Set b
liftS f (S xs) = S (f xs)
```

```
liftS2 :: ([a] -> [b] -> [c]) -> Set a -> Set b -> Set c
liftS2 f (S xs) (S ys) = S (f xs ys)
```

Que outras funcionalidades podem elas permitir? Se exportássemos também estas funções, temos garantia que o seu uso pelo cliente iria sempre garantir as invariantes (ie, lista ordenada sem repetições)?

2. Escreva um módulo que represente um mapa: uma estrutura de dados que mantém associações entre chaves e valores, onde as chaves não devem aparecer repetidas. Defina um tipo de dados para mapas com chaves do tipo k e valores do tipo v .

```
data Map k v = M [(k, v)] deriving Show
```

Utilize uma representação baseada em *listas de associação*, isto é, um mapa deverá ser representado por uma lista de pares chave-valor. Prepare as seguintes operações do módulo, que deverão manter duas invariantes:

- (a) As chaves não aparecem repetidas,
 - (b) A lista está ordenada pelas chaves.
- `empty :: Map k v`, o mapa vazio,
 - `singleton :: k -> v -> Map k v`, construir um mapa com um único elemento,
 - `null :: Map k v -> Bool`, verificar se um mapa está vazio,
 - `size :: Map k v -> Int`, o número de elementos no mapa,
 - `member :: Ord k => k -> Map k v -> Bool`, verificar se uma chave está no mapa,
 - `lookup :: Ord k => k -> Map k v -> Maybe v`, procurar uma chave no mapa, obtendo o valor associado (`Just valor`), ou `Nothing`, caso contrário,
 - `insert :: Ord k => k -> v -> Map k v -> Map k v`, juntar uma entrada (chave, valor) ao mapa, substituindo o valor caso a chave já esteja no mapa,
 - `delete :: Ord k => k -> Map k v -> Map k v`, apagar uma chave e o seu valor de um mapa,
 - `unionWith :: Ord k => (v -> v -> v) -> Map k v -> Map k v -> Map k v`, unir dois mapas, utilizando uma função para combinar os valores de chaves duplicadas,
 - `fromList :: Ord k => [(k, v)] -> Map k v`, criar um mapa a partir de uma lista de pares (não ordenada),

- `toList :: Map k v -> [(k, v)]`, para obter uma lista com os pares constantes num mapa.

Note que o **Prelude** já contém funções com nome **null** e **lookup**, pelo que deverá ter cuidado especial na importação do **Prelude**. Para mais detalhes e exemplos sobre as funções deste exercício, consulte as funções com o mesmo nome no módulo `Data.Map.Strict`.